Лекция 12. Согласованность признаков и Feature Store

Версия: 2026–2027 учебный год Продолжительность: 2 академических часа Связанные работы: ЛР12 «Drift и отложенное качество», ЛР13 «Point-in-time признаки»

1. Цель лекции

Научить студентов формировать исторически корректные признаки и предотвращать training-serving skew. Ключевой результат — понимание, что корректность признака определяется не только значением, но также сущностью, временем события, временем наблюдения, доступностью и версией определения.

2. Результаты обучения

Студент сможет:

- различать event time, feature time, available time и processing time;
- объяснить утечку при latest join;
- построить point-in-time join;
- применить entity key и TTL;
- проверить coverage, feature age и уникальность результата;
- разделить offline historical retrieval и online lookup;
- объяснить назначение entity, data source, feature view и feature service;
- доказать согласованность обучения и inference тестами.

3. Место в сквозном проекте

В ЛР2 команда определила момент прогноза — за 60 минут до планового вылета. В ЛР12 появились временные окна и отложенные labels. Теперь эти правила применяются к вычисленным признакам. Например, trailing delay rate аэропорта полезен только в той версии, которая была известна к моменту прогноза, а не рассчитана задним числом на полном дне.

4. Почему обычный merge опасен

`merge` по `airport_id` отвечает только на вопрос о совпадении сущности. Если после него выбрать последнюю строку истории вообще, рейс утром получит статистику, вычисленную вечером. Модель увидит будущее, validation станет завышенной, а online-сервис не сможет воспроизвести такой вход.

5. Четыре времени

Время	Смысл	Пример
Event time	момент прогнозируемого события	08:00 рейс или момент запроса
Feature time	момент, к которому относится значение	статистика на 07:00
Available time	когда значение стало доступно системе	расчёт завершился в 07:30
Processing time	когда pipeline обрабатывает строку	backfill в 12:00 следующего дня

Feature time может быть раньше event time, но available time позже. В этом случае значение всё равно недоступно для исторического решения.

6. Entity key

Entity key определяет объект, для которого выбирается признак: аэропорт, маршрут, самолёт или клиент. Название сущности и физический join key — разные понятия. Ключ должен иметь стабильную семантику, тип и область уникальности. Смена кода аэропорта требует явной миграции.

7. Point-in-time условие

Для entity event допустима feature row, если:

- entity key совпадает;
- `feature_time <= event_time`;
- `available_time <= event_time`;
- `event_time - feature_time <= TTL`;
- схема и версия определения совместимы.

Из допустимых строк выбирается последняя по feature time, затем по available time как tie-breaker.

8. Наивный и корректный join

Наивный baseline выбирает последнее значение для entity без учёта события. Корректный алгоритм строит кандидатов для каждой entity event, отбрасывает будущее и недоступные строки, применяет TTL, а затем выбирает последнюю допустимую. В ЛР13 оба результата сохраняются, чтобы leakage был виден.

9. Available time и backfill

Предположим, статистика относится к 07:00, но из-за задержки источника сформирована в 09:00. Для прогноза в 08:00 она недоступна. Backfill не должен переписывать историю так, будто значение существовало раньше. `available_time` делает эту задержку наблюдаемой.

10. Created timestamp

В системах Feature Store created timestamp помогает выбрать последнюю физическую версию записи при одинаковом event timestamp. В учебном контракте `available_time` одновременно показывает момент наблюдаемости. В production эти понятия могут разделяться; команда документирует семантику.

11. TTL

TTL ограничивает возраст признака. Слишком короткий TTL создаёт пропуски, слишком длинный допускает устаревшие значения. TTL не делает join корректным сам по себе: значение из будущего остаётся запрещённым независимо от возраста. Порог связывается с частотой обновления и риском stale data.

12. Feature age и coverage

Для каждой строки сохраняется `feature_age_hours`. Coverage — доля entity events, для которых все обязательные признаки найдены. Низкий coverage может означать короткий TTL, пропуски сущностей, задержку materialization или ошибку ключа. Заполнение средним без анализа скрывает причину.

13. Дубликаты и tie-breaking

История признаков должна иметь определённую временную идентичность. Если для entity и timestamp существуют две строки, алгоритм обязан иметь правило выбора или отклонить данные. Недетерминированный join разрушает воспроизводимость даже без явной утечки.

14. Label time не является feature time

Фактическая задержка — отложенная цель. Её нельзя превратить в признак того же рейса. Агрегат по прошлым labels допустим только если окно заканчивается до event time и результат уже доступен. Названия `rolling` и `historical` не гарантируют отсутствие leakage.

15. Training-serving skew

Skew возникает, когда обучение и inference используют разные определения, код, источники, timestamp semantics, defaults или версии. Типичный случай: notebook считает trailing average по полному датасету, а сервис получает последний материализованный показатель.

16. Offline retrieval

Historical retrieval принимает entity dataframe с ключами и timestamps и возвращает значения, корректные для каждого исторического момента. Он используется для training и batch scoring. Результат должен быть версионирован или воспроизводим из определения и исходных данных.

17. Online retrieval

Online lookup получает ключ сущности и обычно возвращает последнее материализованное значение с низкой задержкой. Он не предназначен для построения исторической обучающей выборки: сегодня online store уже содержит знания, которых не существовало в прошлом.

18. Offline-online parity

Parity не означает одинаковый storage engine. Она означает одинаковую семантику и значения для одной версии feature definition в сопоставимый момент. Проверяются имена, типы, units, defaults, transformations и допустимая freshness. Различие требует отчёта, а не молчаливого tolerance.

19. Что такое Feature Store

Feature Store — не просто база признаков. Он сочетает registry определений, retrieval интерфейсы, offline/online источники, materialization и governance. Он не исправляет плохую временную модель автоматически: неверный timestamp в source создаст воспроизводимую, но всё равно ошибочную выборку.

20. Feast Entity и Data Source

В Feast `Entity` задаёт доменную сущность и join key. Data Source указывает, где находятся timestamped feature values. Для исторического retrieval entity dataframe содержит ключи и время, по которому выполняется point-in-time join.

21. Feature View

Feature View группирует признаки, source, entities, schema, TTL и metadata. По официальной документации он используется и для historical datasets, и для загрузки/чтения online values. `enable_validation` помогает проверять наличие и типы столбцов, но не заменяет содержательные тесты.

22. Feature Service

Feature Service объединяет набор feature views или выбранных полей для конкретной версии модели. Это полезная граница совместимости: модель запрашивает логический набор признаков, а release manifest может ссылаться на имя service и registry revision.

23. Feast в плане курса

На 20 октября 2026 года stable release Feast — 0.67.0. В РПД Feast является обзорным или демонстрационным инструментом. Локальная установка не является условием допуска. Обязательны прозрачный pandas join, тест leakage и audit JSON. Feast-демо подтверждает перенос понятий.

24. Definitions as code

Entity, source, feature view и feature service хранятся в Git и проходят review. В definitions указываются owner tags, schema и TTL. Изменение семантики создаёт новую совместимую версию, а не тихую замену поля под прежним именем.

25. Тесты корректности

Минимальный набор:

- никакой `feature_time` не позже `event_time`;
- никакой `available_time` не позже `event_time`;
- feature age не превышает TTL;
- одна строка результата на `request_id`;
- наивный fixture действительно содержит leakage;
- корректный fixture выбирает ожидаемое предыдущее значение;
- coverage соответствует policy.

26. Демонстрация преподавателя

1. Запустить модуль `mlops_flight_delay.features`.
2. Открыть `naive.csv` и показать будущие значения.

3. Сравнить с `point-in-time.csv`.
4. Для события в 08:00 показать feature 07:00, доступную только в 09:00, и её исключение.
5. Уменьшить TTL и объяснить падение coverage.
6. Показать Feast definitions без требования поднимать сервис.

27. Вопросы аудитории

1. Почему `feature_time <= event_time` недостаточно?
2. Чем available time отличается от processing time?
3. Почему online lookup нельзя применять к историческому training set?
4. Что доказывает TTL и чего не доказывает?
5. Как обнаружить training-serving skew?
6. Может ли Feature Store гарантировать отсутствие leakage?
7. Зачем Feature Service связывать с model version?
8. Какой результат считается индивидуальным evidence в ЛР13?

28. Связь с ЛР13 и ЛР14

В ЛР13 команда реализует и защищает временную корректность. В ЛР14 определения признаков, владельцы, ограничения и возможное влияние на группы войдут в model card, data card и review ответственного применения.

29. Официальные источники

- Feast feature retrieval: https://docs.feast.dev/getting-started/concepts/feature-retrieval
- Feast feature view: https://docs.feast.dev/getting-started/concepts/feature-view
- Feast concepts: https://docs.feast.dev/getting-started/concepts
- Feast PyPI: https://pypi.org/project/feast/